# Large Language Models for Opaque Predicate Resolution: A Universal Control Flow Deobfuscation Framework

XIAO CHEN, QIUYUN WANG, SHUWEI WANG, WEIZE ZHANG, YULING LIU, BAOXU LIU, and ZHENGWEI JIANG, Institute of Information Engineering at Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

Code obfuscation is a process that complicates reverse engineering, protects intellectual property, and conceals malware. Existing deobfuscation approaches often lack generality or struggle with complex, mixed, or unknown transformations. To address this issue, this paper proposes LUCID, a Large Language Model (LLM) based Universal Control-flow Integrated Deobfuscation framework. We first formalizes the control-flow deobfuscation task and introduce the Topologically Feasible Path Set (TFPS) as a new evaluation metrics. Building upon this foundation, LUCID leverages an LLM to infer Predicate Mapping Rules between basic blocks in linear time, which then guide the precise expansion of the Runtime Feasible Path Set to identify and eliminate spurious control flows. Finally, semantically equivalent paths are merged to reconstruct a clean, compilable, and behaviorally faithful control-flow graph, from which security-analyst-friendly C-like pseudocode is generated. Comprehensive evaluation on 780 binaries employing 13 distinct obfuscation techniques demonstrates that our method reduces average cyclomatic complexity by 52.4%, achieves full deobfuscation in 53.8% of cases, and suppresses TFPS inflation caused by bogus control flow by over 99%. The framework demonstrates superiority over existing state-of-the-art tools in terms of both generality and semantic consistency, thus evidencing the transformative potential of LLMs in facilitating scalable malware reverse engineering.

CCS Concepts: • **Security and privacy → Software reverse engineering**; • **General and reference → Metrics**; • **Computing methodologies → Information extraction**.

Additional Key Words and Phrases: Deobfuscation, Large Language Model

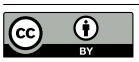
**ACM Reference Format:**
Xiao Chen, Qiuyun Wang, Shuwei Wang, Weize Zhang, Yuling Liu, Baoxu Liu, and Zhengwei Jiang. 2026. Large Language Models for Opaque Predicate Resolution: A Universal Control Flow Deobfuscation Framework. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE092 (July 2026), 23 pages. https://doi.org/10.1145/3808099

## 1 Introduction

Code obfuscation is a widely used protection technique designed to hinder reverse engineering. This technology is commonly utilized for legitimate purposes like IP protection [17], or maliciously by hackers aiming to conceal malware [11]. By transforming the original code into an equivalent but much more complex version, obfuscation intentionally makes it difficult for reverse engineers to understand the program.

A major focus of code obfuscation involves complicating the program's control flow structure. Common methods for achieving this include opaque predicates [12], control-flow flattening, and bogus control flow [31]. These techniques substantially interfere with static analysis and drastically reduce the readability of decompiled code. Because obfuscation increases the difficulty of tasks

Authors' Contact Information: Xiao Chen, chenxiao211@mails.ucas.ac.cn; Qiuyun Wang, wangqiuyun@iie.ac.cn; Shuwei Wang, wangshuwei@iie.ac.cn; Weize Zhang, zhangweize@iie.ac.cn; Yuling Liu, liuyuling@iie.ac.cn; Baoxu Liu, liubaoxu@iie.ac.cn; Zhengwei Jiang, jiangzhengwei@iie.ac.cn, Institute of Information Engineering at Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE092
https://doi.org/10.1145/3808099

like software vulnerability discovery and malware analysis, effective deobfuscation methods are essential for binary analysis and its downstream applications.

Existing deobfuscation tools struggle to provide generic deobfuscation under the realistic assumption that the obfuscation techniques used are unknown. While scripts like D810 [30] and Deflat [14] have been developed to target specific obfuscations, they can be easily circumvented with minor algorithmic modifications. More advanced deobfuscation efforts leverage sophisticated techniques including symbolic execution [5], abstract interpretation [32], program synthesis [54], and satisfiability modulo theories (SMT) solvers [15]. However, past approaches also suffer from a lack of generality: Symbolic execution and abstract interpretation techniques are largely confined to handling data-flow obfuscations, offering limited effectiveness beyond that scope. Abstract interpretation faces challenges in handling the continuous evolution of opaque predicate techniques. SMT solver–based methods, while capable of detecting false paths introduced by obfuscation, are highly sensitive to out-of-distribution code and often fail when confronted with complex conditional structures such as pointer aliasing or Mixed Boolean-Arithmetic (MBA) expressions. More crucially, SMT solvers are ineffective against control-flow flattening, which obscures the true sequential execution path without relying on false predicates.

Leveraging large language models (LLMs) for code deobfuscation has recently emerged as a promising research trend, driven by their potential to offer more general solutions beyond specialized techniques. However, existing LLM-based approaches[4, 6, 7, 9, 37, 46, 51] typically rely on directly generating deobfuscated code, which makes it difficult to ensure functional equivalence between the original and transformed programs. When code appearance changes drastically, reverse engineers also struggle to verify consistency across versions. These issues, despite the promise of LLMs, stem from the lack of a solid theoretical framework for generic deobfuscation.

In light of this, we propose **LUCID**, a LLMs based Universal Control-flow Integrated Deobfuscation framework. LUCID formalizes the core objectives of control-flow deobfuscation and defines the role of LLMs in this task. Guided by heuristic judgments, it intervenes in the decompilation process to reconstruct clear, comprehensible control flow and generate reliable, compilable deobfuscated code.

To evaluate the proposed method, we conducted experiments measuring its deobfuscation effectiveness against state-of-the-art baselines—including rule-based scripts, LLVM's opt, program synthesis models, and LLM direct-interaction solutions. We further tested it against mainstream open-source obfuscators, including Tigress[10], OLLVM[24], and Polaris[61], evaluating efficacy and resilience across diverse control-flow obfuscation techniques.

Our evaluation confirms that the proposed framework establishes a new state of the art in control flow deobfuscation. It achieves a 52.2% reduction in average cyclomatic complexity and completely deobfuscates 53.8% of samples, successfully suppressing over 99% of path set inflation from bogus control flow. The framework demonstrates superior generality across 13 obfuscation techniques and maintains compilable output, effectively addressing the critical challenge of LLM hallucination in binary analysis.

Our contributions include:

- **Formalization of the Control Flow Deobfuscation Task and Metrics**: We formulate the control flow deobfuscation task to guide the integration of LLM and to evaluate both the degree of code obfuscation and the performance of deobfuscation models.
- **A Novel Deobfuscation Framework with Controllable LLM Intervention**: We propose a novel framework for universal control flow deobfuscation. By leveraging linear-time LLM access to derive predicate mapping rules and guide non-linear execution path expansion, we reconstruct concise control flow graphs through equivalent path node merging. This ensures

controllable and trustworthy intervention of LLM in deobfuscation, effectively adapting to diverse obfuscation techniques based on program intrinsic semantics while mitigating hallucination risks.

- **Comprehensive Validation and Benchmarking**: Through extensive benchmarking, we have conducted a comprehensive comparison of our model with other state-of-the-art deobfuscation tools currently available. These tools include script-based, LLM Q&A-based, optimized compilation-based, and program synthesis-based solutions. The test results fully demonstrate that our approach holds a distinct advantage in static deobfuscation analysis.

## 2 Background

### 2.1 Control Flow Obfuscation

Control flow obfuscation is a security technique that masks a program's true execution paths, making it harder for reverse engineers to understand and analyze the code. Based on its core mechanisms and technological evolution, it can be primarily divided into three development directions.

**False Execution Path Injection**. This direction focuses on injecting control-flow branches that are never actually executed to disrupt analysis. Over time, its techniques have evolved from inserting simple NOP instructions to inventing large amounts of junk instructions [11, 27], and further to constructing highly realistic false paths by reusing existing code snippets [3]. Concurrently, the predicates controlling access to these false paths have evolved from simple "always true"/"always false" conditions to arithmetic operations and finally to complex MBA expressions, enabling highly adversarial opaque predicates.

**Decoupling Execution Sequences from Storage Order**. This direction separates the physical layout of code from its execution sequences, undermining the readability of control flow graphs by breaking the correspondence between code structure and execution flow. Key techniques include control flow flattening [10, 25, 31, 57, 61], basic block splitting, loop unrolling [33], code randomization [18] and code virtualization [58].

**Combined Obfuscation**. This direction seeks to maximize code obfuscation by combining multiple techniques. For example, GuiDiv [35] uses entropy-based evaluation to measure the obfuscation strength of randomized code, while Sun, Yongyong [48] employs a knapsack algorithm to balance the intensity and cost of obfuscation algorithm.

### 2.2 Advances in Natural Language Processing for Code Deobfuscation

Recent advances in natural language processing (NLP) have substantially improved the ability of LLMs to understand and generate code [1, 8, 19, 20, 23, 39, 44, 45, 59]. Although most research in this area has focused on high-level languages like C and Python, these techniques have also proven effective for lower-level representations, including LLVM IR [56] and assembly code [49].

This progress has spurred growing exploration of LLMs for code obfuscation and deobfuscation [9, 38, 43, 46]. In high-level language deobfuscation, studies have directly applied LLMs to JavaScript for deobfuscated code generation [6, 7, 37], while others have used LLMs to extract features (e.g., URLs) from obfuscated PowerShell scripts [46]. Similarly, fine-tuning approaches on decompiled pseudo-C or obfuscated source code have been proposed to produce deobfuscated variants [4, 9]. Nevertheless, binary deobfuscation remains significantly more challenging than source-code transformation. Although [51] explored LLM-based assembly code processing for pseudo-C generation, results revealed high error rates in direct code generation.

Beyond binary deobfuscation challenges, a fundamental limitation across previous studies is the frequent absence of safeguards to ensure functional equivalence between original and model-generated code. This often leads to an overreliance on parameter tuning and unrealistic expectations for direct, correct code generation. The problem is further compounded by model illusions [26], which undermines trust in LLMs' outputs. While pattern-based methods like CASCADE attempt to improve reliability through preliminary JavaScript fragment recognition [29], achieving trustworthy deobfuscation remains an open challenge, ultimately requiring manual verification that negates automation benefits.

## 3 Problem Definition

To clarify optimization objectives and guide LLM intervention in control flow deobfuscation pipelines, it is necessary to systematically formalize the processes of control flow obfuscation and deobfuscation.

For any control flow graph $CFG = (V, E)$, two path sets can be defined simultaneously. The first is the Topologically Feasible Path Set $\mathcal{P}_{tfps}$, which visually represents the control-flow graph's topology. The second is the Runtime Feasible Path Set $\mathcal{P}_{rfps}$, comprising only those paths that are executable under concrete runtime conditions. In source code, a well-programmed program will make $\mathcal{P}_{tfps}$ as close as possible to $\mathcal{P}_{rfps}$. This property is generally preserved through compilation: for statically compiled languages like C, the closeness carries over to the machine code level, whereas for interpreted languages like Java, it holds until the intermediate representation (e.g., .class file) level. In contrast, control flow obfuscation deliberately alters $V$ and $E$ to produce a transformed control flow graph $CFG_{obf} = (V', E')$, where $\mathcal{P}_{tfps}(obf)$ includes a substantial number of paths that are not present in $\mathcal{P}_{rfps}(obf)$. Deobfuscation, on the other hand, seeks to reconstruct a minimal topological path set $\mathcal{P}_{tfps}(optim)$ from $CFG_{obf}$ such that all runtime feasible paths are preserved—i.e., $\mathcal{P}_{rfps}(obf) \subseteq \mathcal{P}_{tfps}(optim)$. This optimized set then guides the reconstruction of the final control flow graph.

It is evident that for cyclic graphs, both $\mathcal{P}_{tfps}$ and $\mathcal{P}_{rfps}$ are infinite in size. Consequently, it is not possible to directly calculate the difference set between the two sets. Previous methods of symbolic execution and program synthesis [5] constructed finite $\mathcal{P}_{rfps}$ by restricting the number of loops or the time of execution. In this study, a more theoretical approach is proposed to make it comparable.

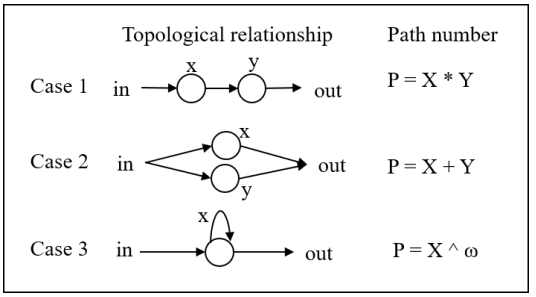



Fig. 1. Three fundamental atomic operations for path counting

Figure 1 shows the three fundamental atomic operations for path counting from node *in* to node *out*. Each node corresponds to a subgraph with the number of paths traversing it denoted $x$ or $y$. The total number of paths from *in* to *out* is described in Case 1 when two nodes are ordered sequentially and in Case 2 when they are ordered side by side. When there exists only one strongly

connected component (SCC) from *in* to *out* with $x$ kinds of basic cycles. While there exist infinitely many types of paths within an SCC, the cycles present in each type can be represented as linear combinations of the basic cycles of SCC. Thus all paths with cycles can be sorted according to the number of basic cycles they contain $\{1, 2, 3, .....\}$. Using the von Neumann ordinal definition, this set corresponds to the smallest infinite ordinal $\omega$. for each $k \in \omega$, there exists a class of subsets of paths, where each subset consists of paths with exactly $k$ basic cycles, and for each basic cycle instance, there are $x$ possible type choices. When considering a finite number of cycles $n$, the total number of paths is positively related to $x^n$, denoted $P \nearrow x^n$. Similarly, for the infinite case, the number of all cyclic paths brought by the entire SCC is described in Case 3, where the index $\omega$ denotes the infinite sequence in the ordinal context.

Formal quantification of the size of TFPS for a CFG employs a theoretical framework based on the *Cycle Aggregation Graph* (CAG) abstraction. Within this model, each SCC of the original CFG is compressed into a single *supernode*. The size of TFPS is defined as Eq. 1.

$$\left|\mathcal{P}_{tfps}(\text{CFG})\right| = \sum_{\pi \in \mathcal{P}_{\text{CAG}}(entry,end)} \left( \prod_{v \in \pi} \left( \alpha_v \times \beta_v^{\omega} \right) \right) \tag{1}$$

where:

- $\mathcal{P}_{\text{CAG}}(entry, end)$ is the set of paths from entry to end in the CAG.
- $\pi = \{v_1, \ldots, v_k\}$ represents a path (sequence of supernodes) in $\mathcal{P}_{\text{CAG}}(entry, end)$.
- For each supernode $v$ (corresponding to $SCC_v$):

$$\alpha_v = \left|\mathcal{P}_{\text{simple}}(v_{in}, v_{out})\right|$$

$$\beta_v = \begin{cases} 1 & \text{if SCC}_v \text{ is acyclic} \\ |e - n + p| & \text{otherwise} \end{cases}$$

  Here, $\alpha_v$ denotes the number of *simple paths* (acyclic paths) between the SCC's entry ($v_{in}$) and exit ($v_{out}$) nodes. $\beta_v$ gives the *cyclomatic number* (cardinality of the cycle basis) for $\text{SCC}_v$, where $e$, $n$, and $p$ represent edge count, node count, and connected components ($p = 1$ for SCCs) respectively.

The cyclomatic number $\beta_v = |e - n + p|$ provides the dimension of the cycle basis for $SCC_v$. This formulation intrinsically handles loop nesting through the exponentiation $\beta_v^{\omega}$.

The scope of this formalization is limited to **control-flow-perturbing obfuscation techniques**, which constitute a subset of all obfuscation methods. Within this scope, the formal model for $P_{tfps}$ size provides a rigorous means to quantify the impact of such obfuscation by analyzing how they alter path complexity in the control-flow graph. Specifically, Bogus Control Flow introduces artificial branches, increasing path complexity linearly via a multiplicative factor on the $\alpha_v$ term. In contrast, Control Flow Flattening restructures $N$ basic blocks into a single SCC governed by a dispatcher, elevating the $\beta_v$ parameter from 1 to $N$ and significantly increasing loop basis cardinality. Similar complexity patterns are observed in virtualization-based obfuscators, where path explosion is induced through dynamic instruction mapping. A further discussion on the generality and limitations of LUCID will be provided in Section 7.

The issue of control flow deobfuscation has now been reduced to two fundamental problems:

- Given a control flow graph $CFG_{obf} = (V, E)$, how to generate its Runtime Feasible Path Set $\mathcal{P}_{rfps}$.
- Given the Runtime Feasible Path Set $\mathcal{P}_{rfps}$, how to generate the minimum control flow graph satisfying Eq. 2. where $\alpha$ is a hyperparameter.

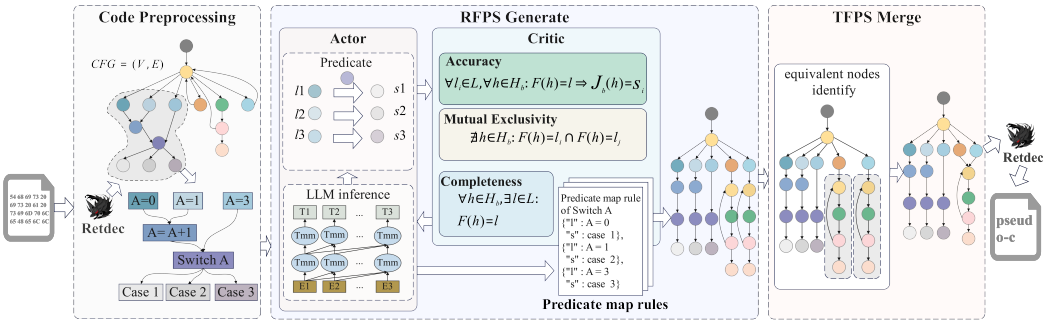



Fig. 2. Overview of LUCID

$$\underset{\mathrm{CFG}_{\mathrm{optim}}=(V',E')}{\operatorname{argmin}} \left\{ \alpha \cdot \left( \left| \mathcal{P}_{\mathrm{tfps}} \right| - \left| \mathcal{P}_{\mathrm{rfps}} \right| \right) + |V'| + |E'| \right\} \tag{2}$$

## 4 Methodology

### 4.1 Overview

Building on the formal analysis of how control-flow obfuscation inflates path complexity, this section details the LUCID framework to solve the two key problems introduced above. Figure 2 illustrates the theoretical model of the three-layer cascade: firstly, the binary is lifted to the intermediate representation (IR) layer; subsequently, the RFPS generation module is injected to recover the execution paths; finally, the deobfuscated control flow graph is reconstructed through the TFPS merge module and fed back to the decompilation process.

### 4.2 RFPS Generator

Reconstructing the exact RFPS is undecidable. Prior work mainly explored two approaches: (1) path constraint solving, which enumerates TFPS paths and uses SMT solvers to verify RFPS membership; and (2) input space sampling, which collects execution paths by feeding diverse inputs. Both methods struggle to recover complete RFPS efficiently—constraint solving suffers from opaque predicates, nonlinear conditions, and path explosion, while sampling is hindered by the vast input space and incomplete coverage.

To mitigate the path explosion problem, we no longer attempt to enumerate every single path, but instead focus on the program's jump relationships. For a given program $P = \{b\}$, the decision function $\mathcal{J}_b$ of basic block $b$ determines the **reachable successor set** $\mathcal{S}(b)$. A path $\pi = \{b_1, b_2, ..., b_n\} \in \mathcal{P}_{tfps}$ does not belong to $\mathcal{P}_{rfps}$ if and only if there exists a basic block $b_{k+1}$ not in $\mathcal{S}(b_k)$. In other words, the decision function $\mathcal{J}$ can completely filter $\mathcal{P}_{tfps}$ to $\mathcal{P}_{rfps}$.

The decision function $\mathcal{J}_b : \mathcal{H}_b \times \mathcal{D} \rightarrow \mathcal{S}_b$ depends on the historical path $\mathcal{H}_b$ (the sequence of blocks from the program start to $b$) and the program input $d$. It is known a priori that the simplification of TFPS should be independent of the program input space $D$. Therefore, the decisions $\mathcal{J}_b(\mathcal{S}_b \mid \mathcal{H}_b, d \in D)$ can be merged for all inputs $d$ in $D$ under the same historical path $\mathcal{H}_b$ into $\phi_b(\mathcal{S}_b \mid \mathcal{H}_b) = \bigcup_{d \in D}\{\mathcal{J}_b(\mathcal{S}_b \mid \mathcal{H}_b, d)\}$. The decision function $\phi$ defines an input-independent superset filter that maps from $\mathcal{P}_{tfps}$ to $\mathcal{P}_{rfps}$.

The decision function $\phi_b$ remains unsolvable directly because its input space consists of the RFPS from the program start to basic block $b$. To address this, all possible historical paths $\mathcal{H}$ are partitioned into equivalence classes based on the decision result $\mathcal{S}$. Under this partitioning, there

must exist a set of basic block sequences $L$ and a mapping function $\mathcal{F}$ such that for any $h \in \mathcal{H}$, $\mathcal{F}(h) = l$ is a valid subsequence of $\mathcal{H}$. Then there is:

$$\forall h_1, h_2 \in \mathcal{H}, \mathcal{F}(h_1) = \mathcal{F}(h_1) \implies \mathcal{S}(h_1) = \mathcal{S}(h_2) \tag{3}$$

Such sequences of basic blocks $l$ are called **conditional prefixes**, and the mapping relation consisting of all the conditional prefixes with the set of reachable successors is called the **predicate mapping rule** (PMR) of the basic block. In the worst case, these conditional prefixes will traverse the RFPS from the program start to basic block $b$.

*4.2.1 Predicate Map Rule Generator.* The generation of PMR for a basic block can be achieved through taint analysis, simulated execution or SMT solving. However, the limitations of these approaches have been previously discussed. In this paper, we employs LLMs for the semantic reasoning of programs, producing heuristic candidates for the PMR. Specifically, given code and the target basic block for analysis, we direct the LLM to generate candidate PMR tuples $\{(\hat{l}_i, \hat{s}_i)\}$, where each $\hat{l}$ represents a generated conditional prefix and $\hat{s}$ corresponding its reachable successor. Figure 2 provides a simple example. The basic blocks A=0, A=1, A=3 are necessary conditional prefixes for Switch A, since they directly determine the subsequent branches that are actually jumped. Conversely, A=A+1 is not a necessary conditional prefix, since later branch behavior in the basic blocks can be distinguished without A=A+1.

The system interfaces with the LLM through LLVM IR, leveraging its superior conciseness over assembly language to reduce input burden while utilizing its structural simplicity compared to high-level languages to ease model comprehension.

To address the LLM illusion problem in PMR generation, we adopt the Actor-Critic architecture to build a verifiable model inference framework. The critic model validates actor's inference outputs at the following three levels: First, **completeness**, $\bigcup l$ must cover all the TFPS reachable paths to $b$. Second, **accuracy**, for all $l_i$, there is no path that simultaneously satisfy $l_i$ but jump to $s_j (s_j \neq s_i)$. Third, **mutual exclusivity**, for all $h \in \mathcal{H}$, $\mathcal{F}(h) = l$ is the unique mapping. Once the validation fails it triggers an iterative correction until the constraint is satisfied or an upper bound is attempted.

*4.2.2 RFPS Unrolling.* This section introduces a path unrolling algorithm (Algorithm 1) based on PMRs, which efficiently constructs execution trees to characterize a program's RFPS without symbolic execution. The algorithm performs a depth-first traversal of the entire CFG (line 3-17). For basic blocks with multiple successors, it applies the mapping function $\mathcal{F}$ to identify conditional prefixes that match the current history path. (line 7) The corresponding successors are then processed as runtime suffixes, rather than their topological successors in the CFG.

The mapping function $\mathcal{F}$ determine if, and where a given sequence of conditions appears as a subsequence within a historical execution path $h$. When multiple matches occur, the rule with the latest matching position is selected, as the influence of prior matches is considered superseded by the most recent one.

Recognizing that the CFG is cyclic, accurate loop identification becomes essential to balance path explosion against obfuscation. We determines loop unrolling based on whether successor branches after the loop are influenced by its body, using a strict unfolding strategy that repeats until external control flow becomes independent of or indistinguishable from the loop. A path segment between two nodes is identified as a valid loop body if, for any successor node $b$ outside the loop, the mapping function $\mathcal{F}$ yields consistent results before and after loop unrolling.

To avoid traversing all nodes backward during each branch node access, we transform the loop detection rule into a comparison between a historical path $h_1$ (with loop body) and a loop-free path $h_2$. The core of this transformation shifts the evaluation from whether loop unrolling alters

the mapping results for specific successor nodes in the execution tree to whether it could change results for abstract successors (which may not exist in the tree). Specifically, for the PMRs of each basic block, we group all prefix subsequences of the conditional prefixes by their remaining suffix subsequences, assuming that these remaining suffixes will be perfectly matched by successor paths outside the loop. We then compute the matching results (via $\mathcal{F}$) for the two historical paths within the same group, and determine whether loop unrolling affects branch selection by checking consistency of the corresponding reachable successor sets. Since this transformation no longer involves actual traversal of successor nodes, the TFPS of the constructed execution tree will be a superset of RFPS. During traversal, if a loop is detected, we construct the back edge of the tree node and terminate expansion of that branch. (line 9-10)

**Algorithm 1** Execution Tree Construction

**Require:** CFG $G$, Predicate map rules $\mathcal{M}$
**Ensure:** Execution tree $\mathcal{T}$

1: $\mathcal{T} \leftarrow \emptyset$ ▷ Initialize tree
2: $Stack \leftarrow \{(\text{root}, \emptyset)\}$ ▷ Stack stores (node, history)
3: **while** $Stack \neq \emptyset$ **do**
4: $(b, \mathcal{H}) \leftarrow \text{pop}(Stack)$
5: $Succ \leftarrow \text{successors}(b, G)$ ▷ Topological feasible successor
6: **if** $|Succ| > 1$ and $\mathcal{M}_b \neq \emptyset$ **then**
7: $Succ \leftarrow \mathcal{M}_b[\mathcal{F}(\mathcal{H}, \mathcal{M}_b)]$ ▷ Runtime feasible successor
8: **end if**
9: **if** has_loop$(b, \mathcal{H})$ **then**
10: linkToAncestor$(\mathcal{T}, \mathcal{H}, b)$
11: **else**
12: addNode$(\mathcal{T}, b, \mathcal{H})$
13: **for** $s \in Succ$ **do**
14: push$(Stack, (s, \mathcal{H} \cup \{b\}))$
15: **end for**
16: **end if**
17: **end while**
18: **return** $\mathcal{T}$

### 4.3 TFPS Merge

The execution tree built in section 4.2.2 has too many nodes for direct use in constructing a control flow graph. In this section we propose an execution tree simplification algorithm based on equivalence class merging, which results in a significant reduction in code size while maintaining functional consistency.

It is observed that when two execution paths share an identical suffix, there exist equivalent nodes on both paths, making both paths indistinguishable after that point. Follow this thought, two nodes $v_1$ and $v_2$ on the execution tree are said to be structurally equivalent (denote as $v_1 \sim v_2$) if the following condition is met:

(1) **Value Identity**: The nodes $v_1$ and $v_2$ represent the same block.
(2) **Migration Isomorphism**: There exists a bijection $\sigma : \partial^+ v_1 \leftrightarrow \partial^+ v_2$ such that for every successor $u \in \partial^+ v_1$, either $\sigma(u) = u$ or $\sigma(u) \sim u$. where $\partial^+ v$ denotes the successor set of node $v$.

(3) **Access Sequence Consistency**: Let $\xi : V \to \mathbb{N}$ map each node to its first-encountered index in a breadth-first search traversal. For every $k \in \mathbb{N}$ exists $\xi(u_k) = \xi(v_k)$, where $u_k$ and $v_k$ are the k-th node encountered in the BFS starting from $v_1$ and $v_2$ respectively.

To prove that these conditions guarantee structural equivalence. We assume for contradiction that $v_1$ and $v_2$ satisfy Conditions 1–3, yet there exists a feasible path $\pi = \langle v_1, u_1, u_2, \ldots, u_n \rangle$ from $v_1$ that is infeasible from $v_2$. By Condition Migration Isomorphism, the bijection $\sigma : \partial^+ v_1 \leftrightarrow \partial^+ v_2$ maps each successor $u_i$ of $v_1$ to $\sigma(u_i) \in \partial^+ v_2$, where $\sigma(u_i) = u_i$ or $\sigma(u_i) \sim u_i$. Applying $\sigma$ recursively to $\pi$ yields $\pi' = \langle v_2, \sigma(u_1), \sigma(u_2), \ldots, \sigma(u_n) \rangle$, which is semantically equivalent to $\pi$. By Condition Access Sequence Consistency, the BFS traversals from $v_1$ and $v_2$ produce identical node indices ($\xi(u_k) = \xi(v_k)$ for all $k$). This, combined with $\sigma$, implies the subtrees rooted at $v_1$ and $v_2$ are isomorphic, so the structural reachability of $\pi'$ from $v_2$ matches that of $\pi$ from $v_1$. Since $\pi$ is feasible from $v_1$, $\pi'$ must be feasible from $v_2$, contradicting the assumption. Thus, no such $\pi$ exists, and $v_1 \sim v_2$ holds.

Equivalence nodes can be identified with the help of transitive closure matrix. Let the comparison matrix $S^k \in \{0,1\}^{n \times n}$ denote the pairs of unjudged nodes remaining after k-step optimisations, which represents all pairs of potentially equivalent nodes. The dynamically updated mask matrix $M^k \in \{0,1\}^{n \times n}$ prunes node pairs that have already been compared. The transitive closure matrix $R \in \{0,1\}^{n \times n}$ captures node reachability and records which nodes will be affected by merges. We iteratively process these three matrices: compare node pairs, update merge effects, and repeat until no more pairs require comparison. The full iterative algorithm is detailed in Algorithm 2.

where:

**Algorithm 2** Control Flow Graph Reconstruction via Equivalence Partitioning

**Require:** Execution tree $\mathcal{T} = (V, E)$ with $|V| = n$
**Ensure:** Simplified CFG $\mathcal{G}'$

Initialize comparison matrix $\mathbf{S} \in \{0,1\}^{n \times n}$
$\mathbf{S}_{ij} \leftarrow [i \geq j] \cdot \delta_{\text{blk}(v_i),\text{blk}(v_j)}$ ▷ $[\cdot]$ denotes Iverson bracket (1 if true, else 0)
Initialize mask matrix $\mathbf{M} \leftarrow \mathbf{S}$ ▷ Mask matrix for fast pruning
$\mathbf{R} \leftarrow$ TransitiveClosure($\mathcal{T}$) ▷ Reachability matrix determines merge-affected nodes
$\mathbf{C} \leftarrow \mathbf{S} \odot \mathbf{M}$ ▷ Active comparison matrix for node pairs to compare
**while** $\|\mathbf{C}\|_F > 0$ **do**
  Select $(i, j)$ where $\mathbf{C}_{ij} = 1$
  **if** ShouldMerge($v_i, v_j$) **then**
    $\mathcal{R} \leftarrow$ ComputeEquivalenceClass($v_i, v_j$) ▷ $v_i$,$v_j$ and their identical suffix
    $\mathcal{T} \leftarrow$ MergeNodes($\mathcal{T}, \mathcal{R}$) ▷ Modify tree: merge nodes in $\mathcal{R}$
    $\mathbf{R} \leftarrow \mathbf{R} - \mathbf{R}_{\mathcal{R},\mathcal{R}}$ ▷ Remove $\mathcal{R}$'s internal edges
    $\mathbf{M}[\mathcal{R}, :] \leftarrow 0; \mathbf{M}[:, \mathcal{R}] \leftarrow 0$ ▷ Prune $\mathcal{R}$ for future comparisons
    $\mathcal{U} \leftarrow \{k \mid \mathbf{R}_{k,\mathcal{R}} \neq \mathbf{0}\}$ ▷ Affected node that can reach $\mathcal{R}$
    $\mathbf{C}[\mathcal{U}, :] \leftarrow \mathbf{S}[\mathcal{U}, :]$ ▷ Update active comparisons for $\mathcal{U}$
    $\mathbf{C} \leftarrow \mathbf{C} \odot \mathbf{M}$ ▷ Eliminate pruned pairs
  **else**
    $\mathbf{C}_{ij} \leftarrow 0$
  **end if**
**end while**
$\mathcal{G}' \leftarrow$ BuildGraph($\mathcal{T}$) ▷ Construct CFG from merged tree
**return** $\mathcal{G}'$

- $\delta_{x,y} = \begin{cases} 1 & x = y \\ 0 & \text{otherwise} \end{cases}$ (Kronecker delta)
- $\text{blk}(v)$ returns basic block of node $v$

## 5 Implementation

The entire implementation was built by python.[1] The decompile process uses the retdec [2] tool. We extended the llvmlite library to make it easier to parse, modify the LLVM IR language and interact with LLMs. For the LLM interaction implementation, we use deepseek-R1 as the generator of the predicate map rule and deepseek-V3 as the critic model to evaluate the model responses.

## 6 Evaluation

In this section, we describe our experimental setup and metrics in detail. Answer the following research questions by comparing with traditional and advanced deobfuscation techniques:

- **RQ1** To what extent does LUCID recover the original control flow graph from obfuscated binaries relative to the ground truth?
- **RQ2** How does LUCID's deobfuscation performance degrade as obfuscation complexity increases, including single versus combined techniques and multiple layers?
- **RQ3** How does LUCID's deobfuscation efficacy compare to state of the art automatic deobfuscation tools on standardized benchmarks?
- **RQ4** What is the individual contribution of each LUCID component to the overall deobfuscation performance in terms of effectiveness and efficiency?

### 6.1 Dataset

To evaluate our approach, we consider two datasets of obfuscated binaries. The first is derived from the Chisel benchmark [40], enabling direct comparison with their method. To further investigate the impact of different obfuscation techniques, implementations, and their combinations, we construct a second dataset from *TheAlgorithms* [50], a high-quality repository of open-source C implementations spanning diverse domains including computer science, mathematics, statistics, data science, machine learning, and engineering.

We begin with 407 source files from this repository. Each is augmented with necessary headers and a synthetic `main` function to enable standalone compilation. After this preprocessing, 291 files compile successfully with GCC. We then apply 13 distinct obfuscation configurations using three state-of-the-art tools: Hikari OLLVM [24], Polaris Obfuscator [61], and Tigress [10]. To avoid source-level bias, we retain only those programs that compile successfully under all 13 configurations. This yields 60 source files, from which we generate 780 obfuscated binaries (13 variants per program).

The Chisel-derived dataset has an average of 23.5 lines of source code per program, while our dataset averages 33.6 lines—reflecting moderately higher algorithmic complexity. The obfuscation configurations encompass five core techniques—Bogus Control Flow (BCF), Control-Flow Flattening (CFF/FLA), Instruction Substitution (Sub), Indirect Branching (IndiBr), and Mixed Boolean Arithmetic (MBA)—along with their pairwise combinations and one composite Tigress transformation, as detailed in Table 1.

### 6.2 Metric

The paper [11] introduces seven possible metrics to measure the degree of code obfuscation, which has been widely used in subsequent work. Since our work focuses on control flow deobfuscation,

[1]The source code is available at https://github.com/IWannaToBeHappy/LUCID.

Table 1. Variant Obfuscation Techniques Configuration

| Tool | Configuration |
|---|---|
| Hikari | BCF, CFF, BCF + CFF, Sub + BCF, Sub + CFF |
| Polaris | BCF, FLA, FLA + IndiBr, FLA + Sub, FLA + MBA, BCF + Sub, BCF + MBA |
| Tigress | Flatten + Indirect + FlattenImplicitFlowNext |

we first select the metrics Cyclomatic Complexity [41] and Nesting Complexity [22] for evaluation. Cyclomatic Complexity is calculated as shown in Eq. 4.

$$CC(G) = e - v + 2p \tag{4}$$

Where $e$ stands for the number of edges, $v$ for the number of nodes, and $p$ for the number of connected components, with $p$ fixed to 1 in the control flow graph.

Nesting Complexity [22] measures code complexity by considering if, for, and while constructs as nested structures and taking the deepest nesting level as the complexity metric. Although the Cyclomatic Complexity metric is good for evaluating the complexity introduced by loops, it cannot express the complexity introduced by branches. Nesting Complexity takes into account the complexity introduced by branching, but it only takes into account the local maximum complexity and not the global complexity.

Recalling the metrics we introduced in section 3 to measure the number of path sets, we can use them to measure the level of obfuscation of a code, specifically, we use Eq. 1 to measure the complexity of a control flow graph.

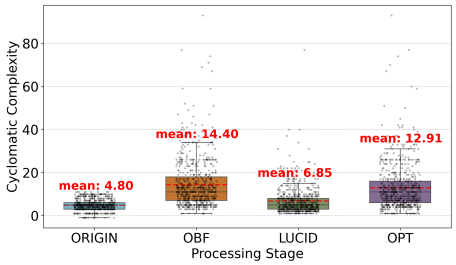
### 6.3 Baseline

To establish a comprehensive and representative baseline, we encompass all four primary categories of methodologies commonly utilized for deobfuscation. The first type of work is the deobfuscation script like deflat and debogus [14], which is widely used to deobfuscate specific obfuscation techniques. The second type of work is the direct use of LLM for deobfuscation. There have been several papers demonstrating that LLM can understand and deobfuscate code [46], we shifted the focus of these tasks from scripting languages like JavaScript to binary files. The third category of work is the traditional program synthesis deobfuscation methods Chisel [40], and the final category of work is the OPT of LLVM project, which has been shown to helpful for code deobfuscation [36].

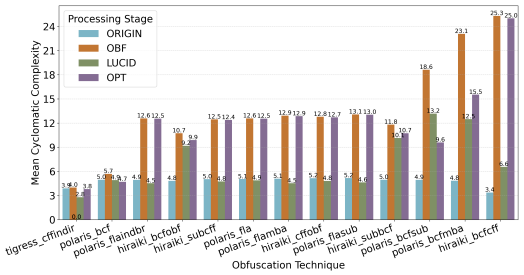
### 6.4 Main Result

This section addresses **RQ 1** and **RQ 2**. Figure 3a and Figure 3c show cyclomatic and nesting complexity distribution across three phases from original to obfuscated to deobfuscated code. The result indicate that obfuscation increases cyclomatic complexity by 200% on average. while LUCID reduces it by 52.2% post-deobfuscation. The median cyclomatic complexity is 5 for original and deobfuscated code, versus 12 for obfuscated code.

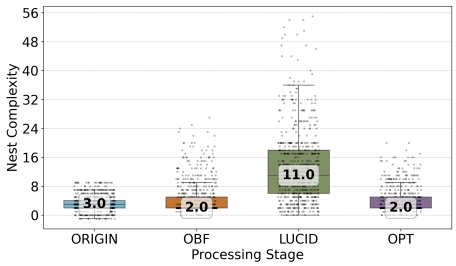
Figure 3b shows that some obfuscations do not increase the cyclomatic complexity, yet LUCID can reduce it below original levels in these cases. CFF notably increases the cyclomatic complexity, and LUCID removes excess cyclomatic complexity in 8/13 cases. LLM-based semantic comprehension enhances resilience of LUCID with MBA/SUB + CFF, though its efficacy weakens slightly for BCF+SUB or MBA hybrids. BCF+CFF greatly raises complexity, yet LUCID restores it to source code levels.



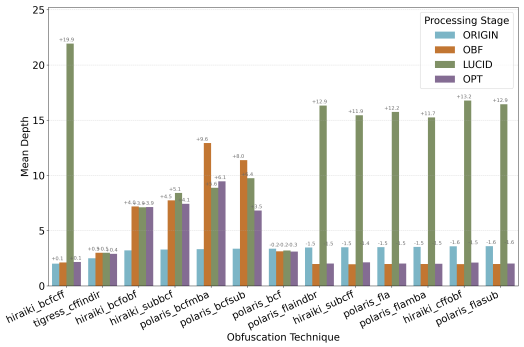

(a) Cyclomatic complexity across stages




(b) Cyclomatic complexity under obfuscation techniques




(c) Nesting complexity across stages




(d) Nesting complexity under obfuscation techniques

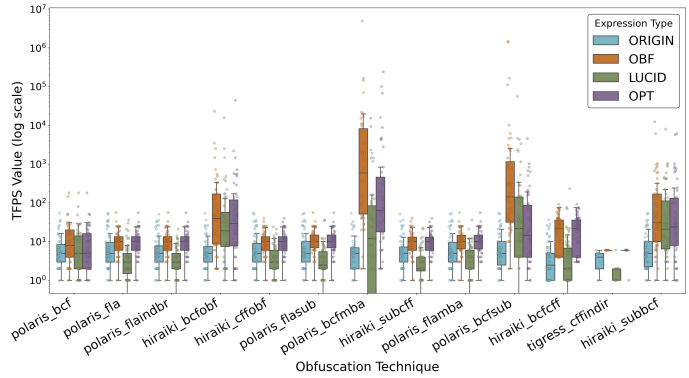
Fig. 3. Comparison of cyclomatic and nesting complexity across stages and obfuscation techniques

Nesting complexity presents a distinct trend. While obfuscated code maintains a median value of 2, comparable to and even lower than the original source code's median of 3, deobfuscated code exhibits significantly higher nesting complexity with a median of 12. This divergence indicates that nesting complexity alone fails to quantify obfuscation severity. Figure 3d demonstrates that 9 of 13 obfuscation techniques do not increase nesting complexity. This is primarily due to the CFF obfuscation, which restructures nested basic blocks into a flat dispatching structures, thereby reducing the overall nesting complexity to a relatively fixed level. Excluding these techniques, LUCID do not increase the nesting complexity of the code. In fact, it reduces the nesting complexity in the polaris_bcfsub and polaris_bcfmba, where the increase in nesting complexity is significant.

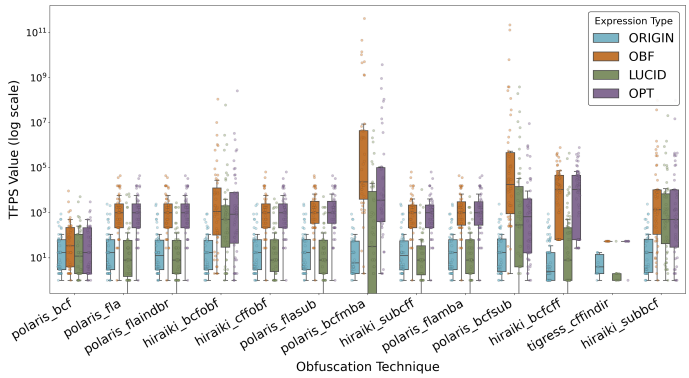
The most comprehensive metric of the code obfuscation level is our proposed **TFPS expression metric**. Figure 4a, 4b shows the TFPS expressions at each stage of the code under various obfuscation techniques, with $\omega$ replacing 1 and 3, respectively. Although $\omega$ formally denotes the smallest infinite ordinal, substituting it with a concrete value enables practical computation of the TFPS over a bounded number of control-flow cycles. This approach allows for a visual comparison of code obfuscation levels. Our evaluation reveals that obfuscated code exhibits substantially higher TFPS values than the original source, confirming the metric's sensitivity to control-flow obfuscation.

> **Answer to RQ1.** LUCID effectively reduces code obfuscation. The deobfuscated output achieves a 52.2% reduction in cyclomatic complexity, and for 53.8% of the evaluated samples, the TFPS metric of the deobfuscated code matches that of the original source code.

Among all 13 obfuscation techniques, 7 obfuscation techniques have a slightly lower distribution of TFPS values ($\omega = 1$) than the source code after deobfuscation, and all optimised codes have




(a) TFPS value with $\omega = 1$




(b) TFPS value with $\omega = 3$

Fig. 4. Values of TFPS expressions under different $\omega$ substitutions across obfuscation techniques

significantly lower TFPS values than the obfuscated code. With $\omega = 1$, BCF causes the largest TFPS increase, CFF leads to a moderate improvement. In contrast, substitution, indirect branching, and MBA techniques show negligible impact on TFPS. This is reasonable because BCF acts on the constant and multiplicative regions of the TFPS expression. CFF act on the base region of the $\omega$ subdimension. Substitution, Indirect branch, and MBA are dataflow obfuscation techniques, which do not alter control-flow structure and therefore have minimal impact. When $\omega = 3$, techniques involving CFF exhibit rapidly growing TFPS values as $\omega$ increases. In contrast, composite techniques built around BCF show a narrowing gap between their obfuscated and deobfuscated TFPS. Notably, for 7 obfuscation techniques, LUCID fully restores the TFPS to match that of the source code, indicating complete removal of obfuscation effects. For the five composite obfuscation techniques with BCF, LUCID simplifies the code complexity but fails to restore it to the source code level.

**Answer to RQ2.** LUCID demonstrates robust deobfuscation performance across diverse obfuscation techniques. For BCF and its variants, it reduces the obfuscation-induced TFPS increase by 98.3%, 99.7%, and 99.9% at $\omega = 1, 2, 3$, respectively. For CFF and its variants, the TFPS values cut the increment from the obfuscation technique by 158.6% (where the optimised TFPS values are lower than the original code), 88.4%, and 85.2% for $\omega = 1, 2, 3$, respectively.

Table 2. Metrics Comparison of LUCID and Debogus Across different Obfuscation Techniques

| Obfuscation | Metric | Origin | Obfuscated | LUCID | Debogus |
|---|---|---|---|---|---|
| | cyc | 3.50 | 6.75 | 2.75 | 6.00 |
| | depth | 2.25 | 1.50 | 10.00 | 1.50 |
| polaris_fla | tfps ($\omega = 1$) | 2.75 | 6.00 | 1.50 | 5.25 |
| | tfps ($\omega = 2$) | 2.75 | 55.50 | 2.00 | 44.75 |
| | tfps ($\omega = 3$) | 2.75 | 619.50 | 3.00 | 470.25 |
| | cyc | 4.00 | 14.20 | 5.20 | 5.00 |
| | depth | 2.50 | 1.60 | 16.00 | 1.20 |
| polaris_flaindbr | tfps ($\omega = 1$) | 3.25 | 13.40 | 5.40 | 4.40 |
| | tfps ($\omega = 2$) | 3.25 | 262.20 | 50.20 | 36.00 |
| | tfps ($\omega = 3$) | 3.25 | 5681.00 | 547.80 | 376.40 |
| | cyc | 3.80 | 10.20 | 3.80 | 5.60 |
| | depth | 2.40 | 1.60 | 13.80 | 1.60 |
| hiraiki_cffobf | tfps ($\omega = 1$) | 3.00 | 9.40 | 3.20 | 4.80 |
| | tfps ($\omega = 2$) | 3.00 | 150.20 | 21.60 | 37.60 |
| | tfps ($\omega = 3$) | 3.00 | 2929.00 | 202.40 | 381.60 |
| | cyc | 4.00 | 11.00 | 4.00 | 5.50 |
| | depth | 2.50 | 1.50 | 14.50 | 1.50 |
| polaris_flasub | tfps ($\omega = 1$) | 3.25 | 10.25 | 3.50 | 4.75 |
| | tfps ($\omega = 2$) | 3.25 | 178.75 | 26.00 | 40.75 |
| | tfps ($\omega = 3$) | 3.25 | 3607.25 | 251.00 | 445.75 |
| | cyc | 4.00 | 11.50 | 4.25 | 6.00 |
| | depth | 2.50 | 1.50 | 15.00 | 1.50 |
| hiraiki_subcff | tfps ($\omega = 1$) | 3.25 | 10.75 | 3.75 | 5.25 |
| | tfps ($\omega = 2$) | 3.25 | 183.75 | 26.75 | 44.75 |
| | tfps ($\omega = 3$) | 3.25 | 3645.25 | 252.75 | 470.25 |
| | cyc | 4.33 | 13.00 | 4.67 | 6.00 |
| | depth | 2.67 | 1.33 | 16.33 | 1.33 |
| polaris_flamba | tfps ($\omega = 1$) | 3.67 | 12.33 | 4.33 | 5.33 |
| | tfps ($\omega = 2$) | 3.67 | 233.00 | 34.33 | 51.33 |
| | tfps ($\omega = 3$) | 3.67 | 4788.33 | 334.33 | 585.33 |
| | cyc | 3.00 | 6.00 | 5.67 | 2.67 |
| | depth | 1.67 | 4.00 | 4.00 | 1.67 |
| hiraiki_subbcf | tfps ($\omega = 1$) | 2.33 | 15.67 | 15.33 | 3.00 |
| | tfps ($\omega = 2$) | 2.33 | 105.67 | 105.33 | 7.00 |
| | tfps ($\omega = 3$) | 2.33 | 783.67 | 783.33 | 19.00 |

## 6.5 Baseline Comparison

*6.5.1 Compare with Script.* Deflat and Debogus[14] are angr-based tools designed specifically for removing CFF and BCF in OLLVM-obfuscated binaries. We apply them directly to our obfuscated samples. Only 19.3% of samples processed by the deflat script and 5.2% by Debogus passed manual verification, highlighting their limited generalizability beyond OLLVM.

Table 3 demonstrates that the deflat script successfully restores code obfuscated via Polaris flattening to a near-original state. The additional use of substitution or MBA techniques does not hinder its deobfuscation effectiveness. Our method also achieves substantial reduction in code

Table 3. Metrics Comparison of LUCID and Deflat Across different Obfuscation Techniques

| Obfuscation | Metric | Origin | Obfuscated | LUCID | Deflat |
|---|---|---|---|---|---|
| | cyc | 5.29 | 11.90 | 4.90 | 5.26 |
| | depth | 3.53 | 2.00 | 15.40 | 3.54 |
| polaris_fla | tfps ($\omega = 1$) | 10.14 | 10.90 | 4.80 | 10.54 |
| | tfps ($\omega = 2$) | 35.31 | 150.95 | 32.57 | 38.21 |
| | tfps ($\omega = 3$) | 178.14 | 2530.76 | 370.38 | 192.97 |
| | cyc | 5.41 | 12.19 | 4.61 | 5.38 |
| | depth | 3.63 | 2.00 | 16.47 | 3.69 |
| polaris_flasub | tfps ($\omega = 1$) | 10.43 | 11.19 | 4.90 | 10.83 |
| | tfps ($\omega = 2$) | 36.20 | 156.66 | 27.57 | 39.07 |
| | tfps ($\omega = 3$) | 180.78 | 2626.76 | 226.90 | 195.54 |
| | cyc | 5.29 | 11.88 | 4.31 | 5.54 |
| | depth | 3.58 | 2.00 | 14.60 | 3.51 |
| polaris_flamba | tfps ($\omega = 1$) | 9.58 | 10.88 | 3.74 | 10.82 |
| | tfps ($\omega = 2$) | 37.35 | 150.08 | 15.51 | 41.80 |
| | tfps ($\omega = 3$) | 203.70 | 2509.97 | 100.94 | 222.88 |

obfuscation. However, due to its deeper nesting structure, the resulting code exhibits a rapid increase in $tfps$ as the $\omega$ substitution value grows.

Debogus scripts cover more sample types than deflat scripts (Table 2), but are implemented in fewer instances due to resource constraints. While the method shows slight superiority under the Polaris_Flaindbr obfuscation technique, it reduces TFPS less effectively than LUCID in five cases. Furthermore, it can restore code obfuscated using Hiraiki_Subbcf to near-original state, exceeding LUCID's performance in this scenario.

*6.5.2 Compare with OPT.* Opt can partially deobfuscate code [21, 36, 60]. To prevent inter-function optimization from disrupting RetDec's function boundary recognition, we apply only the 20 function-internal LLVM optimization passes listed in Table 4. As shown in Figure 3a and Figure 3c, opt's effectiveness lags significantly behind LUCID. The cyclomatic metric shows that opt achieves only 19.8% of the code modification effectiveness compared to LUCID. Figure 3b shows that opt significantly deobfuscates only polaris_bcfsub and polaris_bcfmba techniques. Figure 4a and Figure 4b further show that opt outperforms LUCID only under polaris_bcfsub obfuscation. For almost all flattening obfuscation techniques, opt fails to effectively deobfuscate the code.

Table 4. Optimization Passes Used in Deobfuscation Pipeline

| Category | Pass Name |
|---|---|
| Control Flow | mem2reg, loop-simplify, loop-rotate, loop-unroll-full, tailcallelim |
| Dead Code | dce, bdce, jump-threading, memcpyopt, vector-combine |
| Instruction | instcombine, aggressive-instcombine, instsimplify, reassociate, div-rem-pairs |
| Data Flow | sccp, correlated-propagation, sroa, load-store-vectorizer, dse |

*6.5.3 Compare with LLM.* The success of LLMs in code translation has prompted interest in their use for deobfuscation tasks [46]. However, obfuscated disassembly poses greater challenges than script-based code. When provided with disassembled IR code in a zero-shot setting, the LLM produced compilable outputs in only 9.1% of cases, with the remainder failing due to syntax errors. For the subset that compiled successfully, experts manually verified functional correctness before comparing it against LUCID. As shown in Table 5, the LLM-generated code often exhibits an overly simplified structure, with TFPS base values predominantly at $\omega = 1$ or 2. This suggests that some functionality may have been inadvertently removed.

Overall, pure LLM-based binary code deobfuscation frequently introduces syntax errors, requiring manual verification and thus failing to reduce the reverse engineer's burden. In contrast, LUCID guarantees syntactic accuracy and semantic consistency without manual intervention.

Table 5. Comparison of metrics before and after obfuscated code processing

| Metric | Origin | Obfuscated | LUCID | DeepSeek |
|---|---|---|---|---|
| cyc | 2.69 | 5.63 | 3.37 | 2.03 |
| depth | 1.47 | 2.07 | 4.77 | 1.61 |
| tfps ($\omega = 1$) | 3.23 | 152.59 | 13.56 | 1.9 |
| tfps ($\omega = 2$) | 7.64 | 2314.69 | 233.24 | 3.15 |
| tfps ($\omega = 3$) | 31.15 | 37675.11 | 6112.76 | 6.86 |

Table 6. Compare with chisel

| Metric | Origin | Obfuscated | Chisel | LUCID |
|---|---|---|---|---|
| cyc | 3.02 | 4.25 | 2.60 | 3.29 |
| depth | 1.96 | 2.88 | 1.48 | 3.1 |
| tfps ($\omega = 1$) | 3.0 | 104.3 | 2.48 | 21.38 |
| tfps ($\omega = 2$) | 4.75 | 849.9 | 3.43 | 169.7 |
| tfps ($\omega = 3$) | 10.25 | 7472.1 | 6.10 | 1497.25 |

*6.5.4 Compare with Program Synthesis Method.* Furthermore, we compare our approach with program synthesis–based deobfuscation techniques. Many such methods output only simplified symbolic expressions of memory addresses or register values rather than executable code, leaving a significant gap between their results and fully recovered deobfuscated programs [42]. Among these, Chisel [40] stands out by generating concrete code from obfuscated source code. We evaluate our method against Chisel using its published benchmark.

Chisel operates primarily on source code, but its deobfuscated output often fails to compile even after variable definitions fixes. Overall, While it processes 86% of obfuscated sources, 29.2% of those outputs remain uncompilable, yielding an effective deobfuscation rate of 60.89%. In contrast, our method—working directly from the compiled binaries—successfully deobfuscates 88% of samples with fully compilable and semantically correct results.

Table 6 compares samples deobfuscated by chisel and LUCID. Chisel consistently produces code smaller than the original, eliminating 100% of redundant paths at $\omega = 3$, compare to LUCID's 80.1%. This stems from their differing granularities: Chisel perform at the instruction level, fully unrolling fixed-iteration loops—such as `"for(int i=0; i<10; i++)"`—into long linear instruction

sequences, while LUCID operates at the basic block level and preserves these loop structures. As a result, LUCID reduces obfuscation effectively but achieves less aggressive code compaction. After removing from the benchmark all samples containing such fixed-iteration loops (which account for 20% of the cases), both Chisel and LUCID produce deobfuscated code whose metrics converge to those of the original source. Specifically, the original source has an average cyclomatic complexity of 2.8, nesting complexity of 1.8, and TFPS($\omega = 1$) of 2.7; the obfuscated versions degrade to 3.7, 2.3, and 14.0, respectively. In contrast, LUCID recovers values of 2.9, 2.65, and 2.73, while Chisel achieves 2.4, 1.3, and 2.3—both effectively restoring the program structure to near-original quality.

> **Answer to RQ3.** LUCID achieves the highest end-to-end success rate (88%), substantially outperforming Deflat/Debogus (19.3%), raw LLM prompting (9.1%), and Chisel (60.89%). It produces compilable, functionally correct code and achieves the best TFPS reduction among all binary-level methods. Unlike Chisel—which operates only on source code and aggressively unrolls loops—LUCID works directly on binaries and maintains high fidelity across diverse obfuscation techniques.

### 6.6 Robustness and Efficiency

We assess LUCID's robustness and efficiency through empirical analysis of its core components and runtime behavior.

The critic model plays a pivotal role in ensuring high deobfuscation reliability. Across 7,668 LLM invocations for predicate mapping rule generation, 984 (12.8%) produced syntactically or semantically invalid outputs, and 34 basic blocks required more than five attempts to yield a correct rule. By filtering and correcting these flawed predictions, the critic model elevates the overall success rate from 84.85% (without validation) to 99.49%, demonstrating its necessity for stable performance.

Regarding efficiency, LUCID's design ensures practical scalability. LLM inference is invoked only for multi-successor basic blocks, resulting in linear scaling with program size; processing 840 binaries required 15,370 LLM calls (173M tokens, 61% cached). The TFPS merging algorithm is dominated by transitive closure computation, yielding an $O(N^3)$ worst-case complexity in the number of RFPS nodes $N$. Nevertheless, as shown in Figure 5, merging completes within 10 seconds for most samples ($N < 2{,}000$), and only reaches 204 seconds in an extreme case with $N = 15{,}680$.

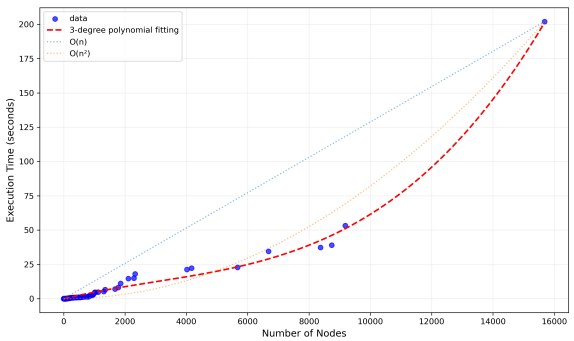



Fig. 5. Relationship between RFPS Tree Size and TFPS merging Time

**Answer to RQ4.** The critic model is essential for high deobfuscation success (99.49% vs. 84.85% without it), while our basic-block-level design and linear LLM usage ensure practical efficiency, with TFPS merging typically completing in under 10 seconds.

### 6.7 Empirical Case

To demonstrate the effectiveness of LUCID in assisting reverse engineering, we analyze a well-known malware sample named Mirai, which has been open-sourced and historically used to infect Internet of Things (IoT) devices for large-scale DDoS attacks, making it a representative and high-value case study. The compiled binary used in our analysis has an MD5 hash of 72cd959f2c31aaf385c0546494ba1568.

We applied control-flow flattening obfuscation to this sample and then deobfuscated it using LUCID, yielding three versions for comparison. In the reverse engineering analysis, we selected the representative malicious function `attack_kill_all` and analyzed its control flow graph. The CFG concisely captures the function's execution logic through a graph structure, clearly highlighting key elements such as the entry (Start) and exit (End) nodes, thereby facilitating structural and semantic understanding.

As illustrated in the Figure 6, the original function exhibits a clear structure: it primarily consists of a loop that repeatedly invokes the malicious subroutine `kill` and exits upon meeting a specific termination condition. After obfuscation, however, this straightforward logic is obscured into a pair of interdependent nested loops, where the iteration count of each loop influences the other. Furthermore, the malicious function `kill` is hidden in an inconspicuous part of the control flow, significantly increasing analytical complexity and hindering rapid reconstruction of the original semantics.

After deobfuscation with LUCID, this multi-loop structure is successfully restored to a single-loop logic consistent with the original function, and the malicious function `kill` clearly appears along the main execution path. By inspecting the pseudo-C code, the intent of the function can be easily determined.

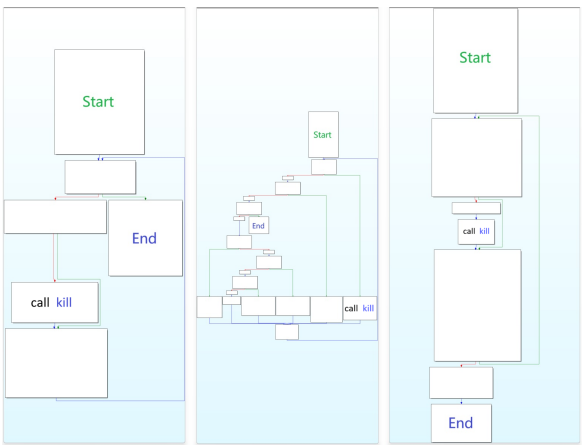



Fig. 6. Control flow graphs of a malicious function in the Mirai sample. From left to right: the original function, the obfuscated function, and the deobfuscated function.

## 7 Discussion

**Implications**. Our framework establishes a reliable LLM integration methodology for binary deobfuscation. Experimental results demonstrate that our approach excels in control flow deobfuscation, exhibits minimal interference from model hallucinations, boasts broad applicability, and offers high scalability. Compared to previous deobfuscation techniques, LUCID ensures the compilability of the output, thereby enhancing its scalability and widening its scope of application. Additionally, we have pioneered a formulaic quantification of control flow obfuscation levels, enabling intuitive evaluation of the impact of various obfuscation and deobfuscation algorithms. Researchers can leverage our deobfuscation algorithm to further optimize control flow outcomes or explore data flow deobfuscation studies.

**Limitations**. The main limitation of our deobfuscation method is its reliance on the decompilation pipeline. When the decompiler fails to reverse the obfuscated binary into pseudo-code or delineate function boundaries, our method will be ineffective. Secondly, our code does not modify symbolic operations within basic blocks, thus its impact on data flow deobfuscation is limited. Finally, experiments show that the control flow graph optimized by the TFPS merge algorithm still has room for further path merging. Given the scalability of our output results, we can employ other methods to further optimize the control flow graph.

Our constructed dataset encompasses five obfuscation techniques as well as their combinations. However, control-flow obfuscation techniques are by no means limited to these five; applying additional tools would introduce further methods such as virtualization obfuscation, loop unrolling, abuse of exception handling, self-modifying code, and others. Exhaustively enumerating all possible obfuscation techniques is inherently difficult. In this section, we discuss the generalizability of LUCID with respect to obfuscation techniques that have not been experimentally evaluated, based on its underlying principles.

Since LUCID explores and simplifies the actual execution path through predicate-based judgment, it can handle techniques that rely on explicit jump logic for obfuscation, including loop unrolling, folding, and similar transformations. When extended with cross-reference analysis of data segments, LUCID can be enhanced to mitigate dynamic data-segment-based obfuscations, such as virtualization obfuscation and dynamically resolved instructions. In particular, for virtualization obfuscation, LUCID effectively replicates a large number of virtual machines via semantic analysis, and redirects the basic blocks corresponding to the actual execution functionality in an attempt to recover the original control flow.

Nevertheless, LUCID remains ineffective against obfuscation techniques that depend on non-explicit jump logic or dynamic program modification, including abuse of exception handling (e.g., INT3 interrupts) and self-modifying code (e.g., overwriting code segment contents during execution). Counteracting such techniques requires dynamic debugging and more sophisticated adversarial analysis.

**Future Works**. Our approach has eliminated false conditional branches in the control flow graph, and our future work primarily focuses on developing more PASSes to further streamline the deobfuscated control flow graph. Firstly, a more sophisticated TFPS merge algorithm could help us simplify the control flow graph and eliminate redundant loops. Secondly, by further decompiling our results into pseudo-C code, we observed a significant number of goto statements, indicating inadequate loop body recognition. This can be addressed by modifying the control flow graph to better align with loop structure identification.

## 8 Related Work

**Program Deobfuscation Research** is a long-standing field of study [55], typically divided into data flow and control flow deobfuscation. While the former simplifies variables and expressions, the latter reconstructs control flow graphs. Since data flow obfuscation can disrupt predicate analysis, both must be addressed jointly. Ramtine Tofighi-Shirazi et al. developed DoSE [54], which established a deobfuscation framework based on semantic equivalence checking. They further proposed combining machine learning with static analysis to address the issue of opaque predicates [52, 53]. Peter Garba et al.[21] proposed SATURN, which builds a deobfuscation framework on top of the LLVM framework. Both have laid the groundwork for subsequent deobfuscation efforts.

**Data Flow Deobfuscation**. Since complex obfuscation techniques such as mixed Boolean arithmetic transformations [62] were proposed, recovering obfuscated variables has attracted extensive research. Aleieldin Salem [47] and others used machine learning to restore metadata information in obfuscated programs. Syntia, proposed by Tim Blazytko et al. [5], pioneered the use of symbolic execution techniques to automatically synthesize the semantics of obfuscated code, and evolved into a series of works such as QSynth [15] and Xyntia [42], which utilize symbolic execution and search algorithms to simplify variable expressions. However, their deobfuscation results mostly consist of computational expressions involving several registers or addresses rather than actual code. This limitation makes the outputs difficult to compare with code recovery-oriented work and also hard for reverse engineers to comprehend.

**Control Flow Deobfuscation**. Research on control flow deobfuscation often targets specific obfuscation techniques. Susmit Jha synthesized loop-free programs via component synthesis [28], while Kevin Coogan [13] and Mingyue Liang [36] applied intermediate language optimizations to restore virtualized code. For OLLVM, Zeliang Kan [32] used symbolic execution and taint analysis. Weiyu Dong [16] proposed CaDeCFF for control flow flattening, and Yuhan Li introduced a more universal multi-architecture anti-OLLVM framework, GOAMD [34]. General control flow deobfuscation algorithms remain an emerging field. Adrian Herrera attempted to use generic optimization to remove obfuscation techniques in JavaScript. Geunha You [60] implemented general deobfuscation optimization in the Android domain using the ReDex optimizer. However, such general optimizations can be easily circumvented by specific obfuscation designs. Mariano B. [40] and others proposed Chisel, which synthesizes programs from symbolic execution results, achieving a more universal control flow deobfuscation.

## 9 Conclusion

We present a general-purpose control flow deobfuscation method to support reverse engineering. By defining novel obfuscation metrics and leveraging LLMs for semantic understanding, our approach addresses interference from complex data flow. Extensive testing across multiple obfuscators shows significant reduction in obfuscation, achieving full deobfuscation for 53.8% of samples. Under loop iteration limits of 1–3, redundant paths from BCF were reduced by up to 99.9%, and from CFF by up to 158.6%. Compared with traditional scripts, opt, LLM Q&A, and program synthesis, our method deobfuscates the most samples while preserving semantics and compilability. It remains scalable and fully compatible with LLVM IR.

## Acknowledgments

This research was also partially supported by Youth Innovation Promotion Association, CAS (No.2023170). Supported by Beijing Key Laboratory of Network Security and Protection Technology.

## References


[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732

[2] Avast. 2025. RetDec. GitHub repository. https://github.com/avast/retdec Accessed: 2025-06-05.

[3] Sebastian Banescu, Christian Collberg, Vijay Ganesh, Zack Newsham, and Alexander Pretschner. 2016. Code obfuscation against symbolic execution attacks. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*. 189–200. doi:10.1145/2991079.2991114

[4] David Beste, Grégoire Menguy, Hossein Hajipour, Mario Fritz, Antonio Emanuele Cinà, Sébastien Bardin, Thorsten Holz, Thorsten Eisenhofer, and Lea Schönherr. 2025. Exploring the Potential of LLMs for Code Deobfuscation. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 267–286. doi:10.1007/978-3-031-97620-9_15

[5] Tim Blazytko, Moritz Contag, Cornelius Aschermann, and Thorsten Holz. 2017. Syntia: Synthesizing the semantics of obfuscated code. In *26th USENIX Security Symposium (USENIX Security 17)*. 643–659. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/blazytko

[6] Luca Caviglione, Carmela Comito, Giacomo Benedetti, Alberto Falcone, and Massimo Guarascio. 2025. Deobfuscation of JavaScript Code and Identification of Security Weaknesses Through Large Language Models. *Future Generation Computer Systems* (2025), 108318. doi:10.1016/j.future.2025.108318

[7] Guoqiang Chen, Xin Jin, and Zhiqiang Lin. 2025. JsDeObsBench: Measuring and Benchmarking LLMs for JavaScript Deobfuscation. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 36–50. doi:10.1145/3719027.3744871

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374

[9] Byunggeon Choi, Hongjoo Jin, Dong Hoon Lee, and Wonsuk Choi. 2025. ChatDEOB: An Effective Deobfuscation Method Based on Large Language Model. In *Information Security Applications*, Jong-Hyouk Lee, Keita Emura, and Sokjoon Lee (Eds.). Springer Nature Singapore, Singapore, 151–163. doi:10.1007/978-981-96-1624-4_12

[10] Christian Collberg. 2023. Flatten. Online. https://tigress.wtf/flatten.html Accessed: 2023-04-10.

[11] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. A taxonomy of obfuscating transformations. https://researchspace.auckland.ac.nz/bitstreams/990cb7de-2b44-4a24-acee-82b0efa8e689/download

[12] Christian Collberg, Clark Thomborson, and Douglas Low. 1998. Manufacturing cheap, resilient, and stealthy opaque constructs. In *Proceedings of the 25th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 184–196. doi:10.1145/268946.268962

[13] Kevin Coogan, Gen Lu, and Saumya Debray. 2011. Deobfuscation of virtualization-obfuscated software: a semantics-based approach. In *Proceedings of the 18th ACM conference on Computer and communications security*. 275–284. doi:10.1145/2046707.2046739

[14] cq674350529. 2025. deflat. GitHub repository. https://github.com/cq674350529/deflat Accessed: 2025-06-05.

[15] Robin David, Luigi Coniglio, Mariano Ceccato, et al. 2020. Qsynth-a program synthesis based approach for binary code deobfuscation. In *BAR 2020 Workshop*. doi:10.14722/bar.2020.23009

[16] Weiyu Dong, Jian Lin, Rui Chang, and Ruimin Wang. 2022. CaDeCFF: Compiler-agnostic deobfuscator of control flow flattening. In *Proceedings of the 13th Asia-Pacific Symposium on Internetware*. 282–291. doi:10.1145/3545258.3545269

[17] Stephen Drape. 2010. Intellectual property protection using obfuscation. (2010). https://www.cs.ox.ac.uk/people/stephen.drape/papers/siemens.pdf

[18] Xuewei Feng, Dongxia Wang, Zhechao Lin, Xiaohui Kuang, and Gang Zhao. 2020. Enhancing randomization entropy of x86-64 code while preserving semantic consistency. In *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 1–12. doi:10.1109/TrustCom50675.2020.00015

[19] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020). doi:10.18653/v1/2020.findings-emnlp.139

[20] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022). doi:10.48550/arXiv.2204.05999

[21] Peter Garba and Matteo Favaro. 2019. Saturn-software deobfuscation framework based on llvm. In *Proceedings of the 3rd ACM Workshop on Software Protection*. 27–38. doi:10.1145/3338503.3357721

[22] Warren A Harrison and Kenneth I Magel. 1981. A complexity measure based on nesting level. *ACM Sigplan Notices* 16, 3 (1981), 63–74. doi:10.1145/947825.947829

[23] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938* (2021). doi:10.48550/arXiv.2105.09938

[24] HikariObfuscator. 2023. Hikari: LLVM Obfuscator. GitHub repository (Archived). https://github.com/HikariObfuscator/Hikari Accessed: 2023-10-26.

[25] Chengyang Li Tianbo Huang, Xiarun Chen, Chenglin Xie, and Weiping Wen. 2022. iOLLVM: Enhanced Version of OLLVM. In *CS & IT Conference Proceedings*, Vol. 12. CS & IT Conference Proceedings. doi:10.5121/csit.2022.120409

[26] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55. doi:10.1145/3703155

[27] Anusthika Jeyashankar. 2022. Most Common Malware Obfuscation Techniques. https://www.socinvestigation.com/most-common-malware-obfuscation-techniques/ Security Investigation.

[28] Susmit Jha, Sumit Gulwani, Sanjit A Seshia, and Ashish Tiwari. 2010. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1*. 215–224. doi:10.1145/1806799.1806833

[29] Shan Jiang, Pranoy Kovuri, David Tao, and Zhixun Tan. 2025. Cascade: Llm-powered javascript deobfuscator at google. *arXiv preprint arXiv:2507.17691* (2025). doi:10.48550/arXiv.2507.17691

[30] joydo. 2020. D810. GitHub repository. https://github.com/joydo/d810 Accessed: 2025-06-05.

[31] Pascal Junod, Julien Rinaldini, Johan Wehrli, and Julie Michielin. 2015. Obfuscator-LLVM–software protection for the masses. In *2015 ieee/acm 1st international workshop on software protection*. IEEE, 3–9. doi:10.1109/SPRO.2015.10

[32] Zeliang Kan, Haoyu Wang, Lei Wu, Yao Guo, and Guoai Xu. 2019. Deobfuscating android native binary code. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 322–323. doi:10.1109/ICSE-Companion.2019.00135

[33] Renuka Kumar and Anjana Mariam Kurian. 2018. A systematic study on static control flow obfuscation techniques in java. *arXiv preprint arXiv:1809.11037* (2018). doi:10.48550/arXiv.1809.11037

[34] Yuhan Li, Bin Wen, and Haixiao Zheng. 2022. Generic O-LLVM Automatic Multi-architecture Deobfuscation Framework Based on Symbolic Execution. In *Proceedings of the 4th International Conference on Advanced Information Science and System*. 1–6. doi:10.1145/3573834.3574541

[35] Yuanpei Li, Qinglei Zhou, Bin Li, and Yan Zhuang. 2023. GuiDiv: Mitigating Code-reuse Attack in an IoT Cluster Using Guided Control Flow Diversification. In *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 422–433. doi:10.1109/TrustCom60117.2023.00073

[36] Mingyue Liang, Zhoujun Li, Qiang Zeng, and Zhejun Fang. 2017. Deobfuscation of virtualization-obfuscated code through symbolic execution and compilation optimization. In *International Conference on Information and Communications Security*. Springer, 313–324. doi:10.1007/978-3-319-89500-0_28

[37] Zhihong Liang, Xin Wang, Zhenhuang Hu, Liangliang Song, Lin Chen, Jingjing Guo, Yanbin Wang, and Ye Tian. 2025. Breaking Obfuscation: Cluster-Aware Graph with LLM-Aided Recovery for Malicious JavaScript Detection. doi:10.48550/arXiv.2507.22447 arXiv:2507.22447 [cs.CR]

[38] Yalan Lin, Chengcheng Wan, Yixiong Fang, and Xiaodong Gu. 2024. CodeCipher: Learning to Obfuscate Source Code Against LLMs. *arXiv preprint arXiv:2410.05797* (2024). doi:10.48550/arXiv.2410.05797

[39] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021). doi:10.48550/arXiv.2102.04664

[40] Benjamin Mariano, Ziteng Wang, Shankara Pailoor, Christian Collberg, and Işil Dillig. 2024. Control-Flow Deobfuscation using Trace-Informed Compositional Program Synthesis. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 2211–2241. doi:10.5281/zenodo.13863694

[41] Thomas J McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320. doi:10.1109/TSE.1976.233837

[42] Grégoire Menguy, Sébastien Bardin, Richard Bonichon, and Cauim de Souza Lima. 2021. Search-based local black-box deobfuscation: understand, improve and mitigate. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2513–2525. doi:10.1145/3460120.3485250

[43] Seyedreza Mohseni, Seyedali Mohammadi, Deepa Tilwani, Yash Saxena, Gerald Ketu Ndawula, Sriram Vema, Edward Raff, and Manas Gaur. 2025. Can LLMs Obfuscate Code? A Systematic Analysis of Large Language Models into Assembly Code Obfuscation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24893–24901. doi:10.1609/aaai.v39i23.34672

[44] Lili Mou, Rui Men, Ge Li, Lu Zhang, and Zhi Jin. 2015. On end-to-end program generation from user intention by deep neural networks. *arXiv preprint arXiv:1510.07211* (2015). doi:10.48550/arXiv.1510.07211

[45] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. A conversational paradigm for program synthesis. *arXiv preprint arXiv:2203.13474* 30 (2022). doi:10.48550/arXiv.2203.13474

[46] Constantinos Patsakis, Fran Casino, and Nikolaos Lykousas. 2024. Assessing LLMs in malicious code deobfuscation of real-world malware campaigns. *Expert Syst. Appl.* 256, C (Dec. 2024), 13 pages. doi:10.1016/j.eswa.2024.124912

[47] Aleieldin Salem and Sebastian Banescu. 2016. Metadata recovery from obfuscated programs using machine learning. In *Proceedings of the 6th workshop on software security, protection, and reverse engineering*. 1–11. doi:10.1145/3015135.3015136

[48] Yongyong Sun. 2018. Software Protection Algorithm based on Control Flow Obfuscation. *International Journal of Performability Engineering* 14, 9 (2018), 2181. doi:10.23940/ijpe.18.09.p27.21812188

[49] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. LLM4Decompile: Decompiling Binary Code with Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 3473–3487. doi:10.18653/v1/2024.emnlp-main.203

[50] TheAlgorithms. 2023. The Algorithms - C. GitHub repository. https://github.com/TheAlgorithms/C Accessed: 2025-06-13.

[51] Anton Tkachenko, Dmitrij Suskevic, and Benjamin Adolphi. 2025. Deconstructing Obfuscation: A four-dimensional framework for evaluating Large Language Models assembly code deobfuscation capabilities. *arXiv preprint arXiv:2505.19887* (2025). doi:10.48550/arXiv.2505.19887

[52] Ramtine Tofighi-Shirazi, Irina Măriuca Asăvoae, and Philippe Elbaz-Vincent. 2019. Fine-grained static detection of obfuscation transforms using ensemble-learning and semantic reasoning. In *Proceedings of the 9th Workshop on Software Security, Protection, and Reverse Engineering*. 1–12. doi:10.1145/3371307.3371313

[53] Ramtine Tofighi-Shirazi, Irina-Mariuca Asavoae, Philippe Elbaz-Vincent, and Thanh-Ha Le. 2019. Defeating opaque predicates statically through machine learning and binary analysis. In *Proceedings of the 3rd ACM Workshop on Software Protection*. 3–14. doi:10.1145/3338503.3357719

[54] Ramtine Tofighi-Shirazi, Maria Christofi, Philippe Elbaz-Vincent, and Thanh-Ha Le. 2018. Dose: Deobfuscation based on semantic equivalence. In *Proceedings of the 8th Software Security, Protection, and Reverse Engineering Workshop*. 1–12. doi:10.1145/3289239.3289243

[55] Sharath K Udupa, Saumya K Debray, and Matias Madou. 2005. Deobfuscation: Reverse engineering obfuscated code. In *12th Working Conference on Reverse Engineering (WCRE'05)*. IEEE, 10–pp. doi:10.1109/WCRE.2005.13

[56] S VenkataKeerthy, Rohit Aggarwal, Shalini Jain, Maunendra Sankar Desarkar, Ramakrishna Upadrasta, and YN Srikant. 2020. Ir2vec: Llvm ir based scalable program embeddings. *ACM Transactions on Architecture and Code Optimization (TACO)* 17, 4 (2020), 1–27. doi:10.1145/3418463

[57] Chenxi Wang, Jonathan Hill, John Knight, and Jack Davidson. 2000. *Software tamper resistance: Obstructing static analysis of programs*. Technical Report. Technical Report CS-2000-12, University of Virginia, 12 2000. doi:10.18130/V36T9V

[58] Wei Wang, Meng Li, Zhanyong Tang, Huanting Wang, Guixin Ye, Fuwei Wang, Jie Ren, Xiaoqing Gong, Dingyi Fang, and Zheng Wang. 2019. Invalidating analysis knowledge for code virtualization protection through partition diversity. *IEEE Access* 7 (2019), 169160–169173. doi:10.1109/ACCESS.2019.2954165

[59] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 8696–8708. doi:10.18653/v1/2021.emnlp-main.685

[60] Geunha You, Gyoosik Kim, Jihyeon Park, Seong-Je Cho, and Minkyu Park. 2020. Reversing obfuscated control flow structures in android apps using redex optimizer. In *The 9th International Conference on Smart Media and Applications*. 272–276. doi:10.1145/3426020.3426089

[61] za233. 2025. Polaris-Obfuscator. GitHub repository. https://github.com/za233/Polaris-Obfuscator Accessed: 2025-06-05.

[62] Yongxin Zhou, Alec Main, Yuan X Gu, and Harold Johnson. 2007. Information hiding in software with mixed boolean-arithmetic transforms. In *International Workshop on Information Security Applications*. Springer, 61–75. doi:10.1007/978-3-540-77535-5_5

Received 2026-02-11; accepted 2026-03-24